

Concurrent Programming TDA384/DIT392

20 March 2026

Exam supervisor: N. Piterman (piterman@chalmers.se, 031 772 6053)

(Exam set by N. Piterman and G. Schneider, based on the courses given in Sep-Oct 2025 and Jan-Mar 2026)

Material permitted during the exam (hjälpmedel):

Two textbooks; four sheets of A4 paper with notes (single or double-sided); English dictionary (no smart phones allowed).

Grading: You can score a maximum of 70 points. Exam grades are: between 28–41 (3), between 42–55 (4), 56 or more (5).

Passing the course requires passing the exam and passing the labs. The overall grade for the course is determined as follows: between 40–59 (3), between 60–79 (4), 80 or more (5).

The exam **results** will be available in Ladok within 15 *working* days after the exam's date.

Instructions and rules:

- Please write your answers clearly and legibly: unnecessarily complicated solutions will lose points, and answers that cannot be read will receive no points!
- Justify your answers, and clearly state any assumptions that your solutions may depend on for correctness.
- Answer each question on a new page. Glance through the whole paper first; four questions, numbered Q1 through Q4. Do not spend more time on any question or part than justified by the points it carries.
- Be precise. In your answers, try to use the programming notation and syntax used in the questions. You can also use pseudo-code, *provided* the meaning is precise and clear. If need be, explain your notation.

This page is left empty intentionally.

WITH ANSWERS!!

Q1 (30p). In what follows you have 10 subquestions (Parts a to j) concerning different topics seen in the course. Each part is a multiple-choice question. The grading for each question is as follows:

- For a right answer you will get 3 points;
- If you do not answer the question, you will get 0 points;
- If you answer wrongly, you will get -1 (a negative point).

The total amount of points will be done summing all the points for each part. So, if you answer correctly Part a, incorrectly Part b, and do not answer any of the rest, you will get 2 points (3 points for Part a minus 1 point for Part b, and 0 for the rest). Note that no negative points for the whole question Q3 will be given (the minimum number of points you may get for the whole question is 0). That is, if you do answer wrongly in each part, you will not get -15 but 0.

(Part a). (3p) According to Amdahl's law, if the fraction p of a program can be parallelized, then, the maximum speedup that can be achieved by n processors is $\frac{1}{(1-p) + \frac{p}{n}}$. Let's assume you have a program where 50% of the program must be done sequentially (and the rest can be parallelized). Which one of the following assertions is *false*?

- The speedup can never be more than double, independently on how many processors you have.
- With 10 processors, you have a speedup less than 2x (around 1.8x).
- There is always a limit on the speedup, independently of the number of processors.
- If you don't know the number of processors, you cannot get a limit on the speedup you can get.
- With 100 processors, the maximum speedup you can get is almost 2x (around 1.98x).

(Part b). (3p) In what follows we state few properties about the pseudocode of the program shown in Fig. 1. Which one is true?

- It doesn't satisfy mutual exclusion, deadlock-freedom nor starvation-freedom.
- It satisfies mutual exclusion, deadlock-freedom and starvation-freedom.
- It doesn't satisfy mutual exclusion but it is deadlock-free and starvation-free.
- It satisfies mutual exclusion and deadlock-freedom, but thread u may starve.
- It satisfies mutual exclusion but the program may deadlock.

Lock lock; Semaphore sem=0; int count=0		
	thread t	thread u
	int c0,c1;	int c0;
1	sem.down();	c0=0; 6
2	lock.lock();	lock.lock(); 7
3	//Critical Section (CS):	sem.up(); 8
4	c0 = count - 1;	//Critical Section (CS) 9
5	c1 = c0 * 2;	count = c0 + 10; 10
6	count = c0 + c1 + 2;	c0 = count + 2; 11
7	//End CS	//End CS 12
8	lock.unlock();	sem.down(); 13
9	sem.down();	lock.unlock(); 14

Figure 1: Q1-b): Pseudocode of program someCounting.

(Part c). (3p) See below the pseudocode of two threads **t** and **u**, where the function **randomN()** gives a random negative number:

bool flag = true; int value = 1;		
	thread t	thread u
1	if (! flag)	value = randomN(); 3
2	print(value);	flag = false; 4

Assume that a programmer has implemented the pseudocode in her favorite programming language and executes it. Assume the threads are called 10 billion times, where both threads have finished their execution in each *round* (i.e., a new round only starts when both threads are finished). Which one of the following assertions is true?

- The program never prints anything, independently of the programming language used.
- The program always print the value of **value** in each execution, independently of the programming language used.
- The program never prints 1, independently of the programming language used.
- The outcome depends on the programming language used and its underlying memory model. It might sometimes print 1, sometimes print a negative number, sometimes print nothing.

e) None of the above is true.

(Part d). (3p) A solution to concurrently access a list is to use a coarse-locking method, locking the entire list. In a lecture, we have seen that though this works it is not satisfactory since the access is essentially sequential, and we gave different alternative approaches. This question is about fine-grained locking, that is, rather than locking the whole linked list at once, we add a lock to each node (threads only lock the individual nodes on which they are operating). Which one of the following assertion about fine-grained locking is true?

- a) To guarantee that the concurrent access works well (i.e., there are no inconsistencies), it is enough that both threads lock only their **pred** pointed node when executing the **find** method.
- b) The order in which you lock the **pred** and the **succ** nodes are irrelevant in the implementation of the **find** method.
- c) If too many threads execute the validation process simultaneously (to ensure no two threads are accessing the same node at the same time), the fine-grained locking does not work and inconsistencies may arise.
- d) The *hand-over-hand locking* (also known as *lock coupling*) ensures the locks are acquired in the right order and helps to avoid deadlock.
- e) None of the above is true.

(Part e). (3p)

This is Peterson's algorithm for 2 threads:

boolean[] enter = {false, false}; int yield = 0 1;	
thread t_0	thread t_1
<pre> 1 while (true) { 2 // entry protocol 3 enter[0] = true; 4 yield = 0; 5 await (!enter[1] yield != 0); 6 critical section { ... } 7 // exit protocol 8 enter[0] = false; 9 }</pre>	<pre> 10 while (true) { 11 // entry protocol 12 enter[1] = true; 13 yield = 1; 14 await (!enter[0] yield != 1); 15 critical section { ... } 16 // exit protocol 17 enter[1] = false; 18 }</pre>

What of the following sentences about Peterson's algorithm is true?

- a) The algorithm is still correct if line 3 is replaced by **enter[1] = true**, and line 12 is replaced by **enter[0] = true**.

- b) The algorithm is still correct if line 8 is replaced by `enter[1] = false`, and line 17 is replaced by `enter[0] = false`.
- c) The algorithm is still correct if you perform all the replacements indicated in the first two options above.
- d) The algorithm introduces a deadlock if you perform all the replacements indicated in the first two options above.
- e) None of the above is true.

(Part f). (3p)

We aim at implementing a semaphore using `wait()` and `notify()`. The following is an implementation of the `up()` operation of a semaphore in Java:

```
public synchronized void up()
{
    ++count;
    notify();
}
```

These are four possible implementations of `down()`:

```
public synchronized void down1()
    throw InterruptedException
{
    wait();
    count = count-1;
}
```

```
public synchronized void down2()
    throw InterruptedException
{
    if (count == 0) { wait(); }
    count = count-1;
}
```

```
public synchronized void down3()
    throw InterruptedException
{
    while (count == 0) { wait(); }
    count = count-1;
}
```

```
public void down4()
    throw InterruptedException
{
    while (count == 0) { wait(); }
    count = count-1;
}
```

Which one of the following assertions is true?

- a) `down1()` is a correct implementation of `down()`.
- b) `down2()` is a correct implementation of `down()`.
- c) `down3()` is a correct implementation of `down()`.
- d) `down4()` is a correct implementation of `down()`.
- e) None of the above is true.

(Part g). (3p)

See below an implementation of the `add` function of a Sequential Set data structure as seen in the lectures. The set stores items in increasing key order. The `find` function returns a pair of nodes *pred* and *curr* such that $pred.key() < node.key() \leq curr.key()$.

```

public boolean add(T item) {
    Node<T> node = new Node(item);
    Node<T> pred = null, curr = null;
    (pred, curr) = find(head, node.key());
    if (curr.key() == node.key())
        return false; // item already in set
    else { // insert node between pred and curr
        node.setNext(curr);
        pred.setNext(node);
        return true;
    }
}

```

Which one of the following options is true?

- a) The implementation is thread-safe (i.e., no inconsistencies arise when multiple threads operates on the data structure simultaneously).
- b) We cannot say anything on whether the implementation is thread-safe since we need to know how the other operations on the data structure are implemented.
- c) The implementation is not thread-safe since threads can add nodes in between two adjacent nodes and create an inconsistency.
- d) The implementation is thread-safe only if the operation `find` is thread-free.
- e) None of the above is true.

(Part h). (3p) Consider the following Erlang server:

```

server(Value) ->
receive
    {inc} ->      server(Value + 1);
    {get, From} -> From ! {value, Value},
                    server(Value)
end.

```

Two identical clients interact with the server concurrently:

```

Server ! {inc},
Server ! {get, self()},
receive
    {value, V} -> V
end.

```

Assume the server initially runs as `server(0)` and both clients execute concurrently.

Which one of the following assertions is true about the final value stored by the server after both clients terminate?

- a) The server value is always 0.
- b) The server value is always 1.
- c) The server value is always 2.
- d) The server value may be either 1 or 2 depending on the order the server processes messages.
- e) None of the above is true.

(Part i). (3p)

Revisiting the same Erlang server in part (h). Which one of the following assertions is true about the values that the two clients may receive?

- a) Both clients always receive 1.
- b) Both clients always receive 2.
- c) One client always receives 1 and the other receives 2.
- d) Each client may receive either 1 or 2, depending on the order the server processes messages.
- e) None of the above is true.

(Part j). (3p) Consider the following Java class:

```
class Pair {
    private Integer x = 0;
    private Integer y = 0;

    public void addToBoth() {
        synchronized (x) {
            x++;
        }
        synchronized (y) {
            y++;
        }
    }

    public boolean equal() {
        synchronized (y) {
            synchronized (x) {
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16


```

        return x.equals(y);
    }
}
}
}

```

17
18
19
20
21

Which statement is correct?

- a) Every call to `equal()` will return true.
- b) `equal()` may return false but deadlock is impossible.
- c) The class may deadlock.
- d) The code is wrong because it synchronizes on different objects.
- e) None of the above is true.

Answers:

(Part a). *Option d: It is false since you can always set a limit on the speedup if you know how much parallelism you can get (in the worst case, the speedup cannot be more than if you assume there is an infinite number of processors; in the current case, the limit is 2).*

(Part b). *Option e: Only one thread at a time may be in the CS given the use of the lock. That said, only thread u can acquire the lock at the beginning (since thread t cannot execute the down operation) and if after u executing `sem.up()`, thread t executes `sem.down()` then thread u won't be able to execute `sem.down()` in which case a deadlock occurs (thread u gets stuck trying to execute down (line 12) and thread t) is stuck waiting for the lock (line 2)).*

(Part c). *Option d: That is the output if you use Java (as we have seen, Java has a Weak Memory Model and unless you take care to use synchronization mechanism the program might eventually give 1 as answer).*

(Part d). *Option d: This is indeed the case, as this is exactly how find is implemented (it uses the hand-over-hand protocol).*

(Part e). *Option c: Yes, it still guarantees mutual exclusion, deadlock freedom and starvation freedom. The only change is that instead of saying "I want to enter into the critical section" (by setting `enter[i]=true` for i equal 0 or 1 depending on the thread), it says "You want to enter into the critical section" (by inverting the value of i).*

To see that it guarantees deadlock freedom see that the await condition means that at any moment one of the threads (and only one) will be able to enter the critical section since `yield` can be 0 or 1 but not both at the same time (and it will be 0 or 1 in all iterations).

Note that there is mutual exclusion since there is no way to have `yield=0` and `enter[0]= false` (similarly, not possible that `yield=1` and `enter[1]= false`). So, at any moment only one of the threads would be able to go in.

(Part f). Option c: `down1` is not correct since the decrement is done unconditionally. `down2` is not correct since the check that count is zero is done conditionally, but the value of count might change between the checking and the execution of wait (it should be under a while as in `down4`). `down4` is wrong since the method is not synchronized.

(Part g). Option c: This is indeed the case as the implementation is not thread safe. For example, if the list contains elements with keys 1 and 4, and two threads try to add 2 and 3 simultaneously, one addition could over-write the other. In particular, if both execute the line `pred.setNext(node)` one after the other, the second will over-write the first.

(Part h). Option c: The server will have received two `inc` messages, it will have processed them and incremented the value to 2.

(Part i). Option e: It is impossible for both clients to receive the answer 1. For one client the `get` reaches the server after the `inc` as they are sent from the same process. Then, they will get 1 if the `inc` of the other process reaches the server after it has processed the `get`. Otherwise, they will get 2.

In general, the order of the messages received by the server is any interleaving of two `inc` and `get`. The server handles `inc` before it handles `get`:

- `inc1, get1, inc2, get2` leads to either client 1 getting 1 or both clients getting 2.
- `inc1, inc2, get1, get2` leads to both clients getting 2.
- `inc1, inc2, get2, get1` leads to both clients getting 2.

There are three similar scenarios starting with `inc2`.

(Part j). Option b: deadlock is impossible as all threads that lock and hold lock `y` first and then `x`. So there cannot be a circular dependency. On the other hand, while one thread calls `equal()` and locks `y`, another thread may increment `x` and then be blocked from incrementing `y` until the thread checking equality will have already concluded `unequal`.

Q2 (12p). In a car production line, robots produce brake pedals and put them on a circular belt. The pedals are taken by other robots along the production line and pass it on to be assembled in the car. Brake producer robots are called *makers* and the ones taking the brakes from the belt are called *users*.

The belt has a capacity **Max**, that is, it is not possible to add more pedals if the belt is full. (**Max** is strictly positive, i.e., **Max**>0) Similarly, no user can take a pedal from the belt if this is empty.

You can assume that there are m makers and n users ($n, m > 0$).

A programmer with no experience in concurrency, was in charge of writing a program to handle the above with the explicit instruction that the program should be *deadlock-free*, *starvation-free* and it should work for an *arbitrary number of makers and users*. The programmer wrote the code in Fig. 2 defining a class **Belt** and two methods for putting pedals in the belt (**put**) and taking pedals from the belt (**take**). Note: **belt** is defined as: **Belt**<pedal> **belt**.

```
public class Belt<T> {
    Semaphore nPedals = new Semaphore(0);
    Semaphore nFree = new Semaphore(Max);

    Lock lock = new Lock();
    Collection store = ...; // implem. of a collection of pedals

1 public void put(T pedal) {
2     nFree.down();
3     lock.lock();
4     store.add(pedal);
5     lock.unlock();
6     nPedals.up();
7 }

8 public T take() {
9     lock.lock();
10    nPedals.down();
11    T pedal = store.remove();
12    nFree.up();
13    lock.unlock();
14    return pedal;
15 }
```

Figure 2: Q2: Car production code for makers and users.

Besides the shown code, there is a **main** program that creates m threads for makers and n threads for users. Each maker keeps creating pedals and putting them into the belt by calling **belt.put(pedal)** where users keep taking them from the belt by calling **pedal = belt.take()** and then passing them to the rest of the production line.

In what follows there are five assertions. State whether they are true or false and justify your answer in each case. An answer without justification will not get full points.

- 1 The program works incorrectly only if the number of users is larger than the number of makers. (2p)

- 2 The program is incorrect since in the method `put`, the execution of `lock.lock()` (line 3) should happen before `nFree.down()` (line 2). **(2p)**
- 3 The program is incorrect since in the method `put`, the execution of `nPedals.up()` (line 6) should happen before the call to `lock.unlock()` (line 5). **(2p)**
- 4 The program is incorrect since in the method `take`, the execution of `nPedals.down()` (line 10) should happen before the call to `lock.lock()` (line 9). **(2p)**
- 5 The program is correct (that is, it is deadlock-free, starvation-free, and it works for an arbitrary number of makers and users). **(2p)**
- 6 A correct solution of the program does not necessarily require the use of two semaphore and one lock: deadlock-freedom, starvation-freedom and mutual exclusion can be achieved with three semaphores (replacing the lock with a semaphore). **(2p)**

ANSWER:

- 1 *False: The program is not correct but not because of the number of makers and users (the number of processes do not affect the correctness of the program).*
- 2 *False: the program is incorrect but swapping those two lines in put may introduce a deadlock.*
- 3 *False: the program is incorrect but swapping those two lines won't solve the problem as swapping lines 5 and 6 will still preserve the semantics of the program.*
- 4 *True: As it is the program will deadlock exactly because of the order of these two lines in take.*
- 5 *False: As show in previous assertion, the program may deadlock.*
- 6 *True: This is a general truth: a semaphore is a generalization of a lock so the latter can always be replaced by the former.*

Q3 (13p). Consider the following Erlang code implementing a simple mutex (lock) server.

```

-module(lock).
-export([start/0, lock/1, unlock/1]).

start() ->
    spawn(fun() -> loop(false, []) end).

lock(Pid) ->
    Ref = make_ref(),
    Pid ! {lock, self(), Ref},
    receive
        {granted, Ref} -> ok
    end.

unlock(Pid) ->
    Pid ! unlock.

loop(Locked, Waiting) ->
    receive
        {lock, From, Ref} ->
            case Locked of
                false ->
                    From ! {granted, Ref},
                    loop(true, Waiting);
                true ->
                    loop(true, [{From, Ref} | Waiting])
            end;
        unlock ->
            case Waiting of
                [] ->
                    loop(false, []);
                [{P,R}|Rest] ->
                    P ! {granted, R},
                    loop(true, Rest)
            end
    end.
end.

```

(Part a). (3p)

Does this server implement the interface promises of a lock?

(Part b). (4p)

Analyze the behavior of a system where multiple clients are running the following code and interacting with the lock server above.

```

critical_section(LockPid, NcsFun, CsFun, State) -> 1
    State1 = NcsFun(State), %% Run non-critical section and get new state 2
    lock(LockPid), 3
    State2 = CsFun(State1), %% Run critical section and get new state 4
    unlock(LockPid), 5
    critical_section(LockPid, NcsFun, CsFun, State2). 6
7
8

```

What are the guarantees supplied by this system and what are the guarantees not supplied?

(Part c). (3p)

Suggest a modification to the server that solves the problems identified in part (a).

(Part d). (3p)

Suggest a modification to the server that solves the problems identified in part (b).

Answer:

(Part a).

The implementation indeed ensures that at most one process can take the lock at a time. However, it does not ensure that `unlock()` is only allowed to be called by a process that has already taken the lock.

(Part b).

With this usage scenario, processes only unlock when they have taken the lock before. So the problem in (a) does not affect the behavior. The combination of the server and the clients ensures mutual exclusion and deadlock freedom. But because the list of waiting is maintained as a LIFO, starvation is possible.

(Part c+d).

In order to ensure that only the locking process unlocks, the server needs to record some information about the holder of the lock. Either the PID or a PID and a reference that will be used for unlock. The simple solution to (d) is to replace line 25 by: `loop(true, Waiting ++ [From,Ref])`. A more elegant solution is to leave the lock messages in the mailbox. Here is one possible integrated solution.

```

1 lock_fixed(Pid) ->
2   Ref = make_ref(),
3   Pid ! {lock, self(), Ref},
4   receive
5     {granted, Ref} -> ok

```

```

6  end
7  Ref.
8
9  unlock_fixed(Pid, Ref) ->
10  Pid ! {unlock, self(), Ref},
11  receive
12    {granted, Ref} -> ok;
13    {error, not_owner} -> throw({error, not_owner})
14  end.

```

The lock function now returns a reference and the unlock may throw an exception.

```

1  critical_section_fixed(LockPid, NcsFun, CsFun, State) ->
2  State1 = NcsFun(State), %% Run non-critical section and get new state
3
4  Ref = lock_fixed(LockPid),
5  State2 = CsFun(State1), %% Run critical section and get new state
6  unlock_fixed(LockPid, Ref),
7
8  critical_section(LockPid, NcsFun, CsFun, State2).

```

The usage scenario now saves the returned reference and uses it for unlocking.

```

1  loop_fixed(false) ->
2  receive
3    {lock, From, Ref} ->
4      From ! {granted, Ref},
5      loop_fixed(true, Ref);
6  end.
7
8  loop_fixed(true, Owner, Ref) ->
9  receive
10   {unlock, Owner, Ref} ->
11     From ! {granted, Ref},
12     loop_fixed(false);
13   {unlock, _, _} ->
14     From ! {error, not_owner},
15     loop_fixed(true, Owner, Ref);
16  end.

```

The loop now leaves locking requests on the mailbox. This means that locking requests are handled in the order that they are received. It further distinguishes between legal and illegal unlocks.

Q4 (15p). We present several programs solving synchronization problems. For each program you need to analyze whether and how well it solves the respective problem, and provide an explanation about your analysis and conclusion.

(Part a). (6p) The following program implements a barrier.

```
import java.util.concurrent.Semaphore;

class MyBarrier {
    private final int N;
    private final Semaphore counter;
    private final Semaphore gate;

    public MyBarrier(int N) {
        this.N = N;
        this.counter = new Semaphore(N);
        this.gate = new Semaphore(0);
    }

    public void await() throws InterruptedException {
        int before = counter.availablePermits();
        counter.acquire();
        int after = counter.availablePermits();

        boolean iAmLast = (before == 1 && after == 0);

        if (iAmLast) {
            counter.release(N);
            gate.release(N);
        }

        gate.acquire();
    }
}
```

(Part b). (4p) The following code is a monitor managing the access to two identical resources. Threads call **acquire()** before using a resource and **release()** when finished. Analyze the guarantees of this program.

```
class TwoResources {
    private int available = 2;

    public synchronized void acquire() throws InterruptedException {
```



```

        while (available == 0)
            wait();
        available--;
    }

    public synchronized void release() {
        available++;
        notify();
    }
}

```

(Part c). (5p) The following program implements a workshop with two tools. Before starting work, a thread requires to have the two available tools. We are ensured that every thread before starting work will call acquire to both tools.

```

import java.util.concurrent.locks.*;

class Workshop {
    private int hammers;
    private int screwdrivers;

    private final Lock lock = new ReentrantLock();
    private final Condition hammerAvailable = lock.newCondition();
    private final Condition screwdriverAvailable = lock.newCondition();

    public Workshop(int h, int s) {
        hammers = h;
        screwdrivers = s;
    }

    public void acquireHammer() throws InterruptedException {
        lock.lock();
        try {
            while (hammers == 0)
                hammerAvailable.await();
            hammers--;
        } finally {
            lock.unlock();
        }
    }

    public void acquireScrewdriver() throws InterruptedException {
        lock.lock();
    }
}

```

```

    try {
        while (screwdrivers == 0)
            screwdriverAvailable.await();
        screwdrivers--;
    } finally {
        lock.unlock();
    }
}

public void releaseHammer() {
    lock.lock();
    try {
        hammers++;
        hammerAvailable.signal();
    } finally {
        lock.unlock();
    }
}

public void releaseScrewdriver() {
    lock.lock();
    try {
        screwdrivers++;
        screwdriverAvailable.signal();
    } finally {
        lock.unlock();
    }
}
}

```

Answers:

(Part a)

The calls to `counter.availablePermits()` are not synchronized. This means that there is no assurance that the value **before** and the value **after** correspond to the thread's own actions. In particular, it could be the case that no thread perceives that they are the last and the entire system gets deadlocked. It is not possible for multiple threads to think that they are last.

(Part b)

This works fine. It ensures that no more than two threads access the resources (a variant of mutual exclusion), it ensures deadlock freedom, but it does not ensure lack of starvation as the locks used by synchronized are not fair.

(Part c)

The program ensures that screwdrivers and hammers are allocated as required (a variant of mutual exclusion). However, if threads acquire in different orders they could have a cyclic dependency and end up in a deadlock. Given that the system can deadlock it is definitely not starvation free.